AZAM-PNET ENTERPRISE PLATFORM • SYSTEM DOCUMENTATION

Enterprise Features & Architecture Operations Manual

A Comprehensive Reference to Dual-Plane Master & Satellite Taxonomy, UI Placements, Operator Workflows, and Technical Background Lifecycles

Target System: AzamLabs / AzamLabs Enterprise	**Document Version:** 2.7.0-LTS (Release 6.8.93resolute1)
Generated: November 02, 2026	**Architectural Planes:** Master Control Plane + Satellite Compute Plane
System Daemons: azambasha-ops-api (:8088), azam-watchdog.service	**Security & Role Guard:** RBAC Admin Guard (Role 0) + Airgap Packaging

1. Executive Architectural Taxonomy & Feature Placements

AzamLabs organizes all enterprise capabilities across three deliberate user and runtime environments: **Outside-the-Lab Canvas** (global hypervisor diagnostics, template repository marketplace, multi-cloud and offline air-gapped backups, capacity planning, golden image auditing, idle scheduler, client toolkit generator, and SSL automation accessed from the main administrative dashboard), **Inside-the-Lab Canvas** (in-workbench workflow tools, live traffic bandwidth heatmaps, WAN QoS link impairment, multi-node snapshots, RESTCONF sandbox, Chaos Monkey link flaps, NetDevOps inventory export, CFS vCPU throttling, AI copilot drawer, anti-bootstorm staggered startup with console ready-state probing, Git version control, and live Wireshark packet capture available right inside the active topology view), and **Autonomous Kernel & Background Daemons** (24/7 self-healing watchdogs, Soft-RoCE RXE MTU 9000 kernel engines, MySQL socket auto-healers, and nightly SSD TRIM cron jobs running continuously in the host background).

Feature Name	In GUI?	Architectural Zone	Exact UI / System Location	Primary Trigger	Plane Support
1. Live Hot-Node Profiler (RBAC Guard)	Yes	Outside Lab	Main Nav > Azam Features > Health & Diagnostics	Real-Time / Kill	Master & CLI
2. HTML5 Console Session Reset	Yes	Outside Lab	Main Nav > Azam Features > Console Fixer	Repair Button	Master Only
3. Fleet Multi-Node Cluster Monitor	Yes	Outside Lab	Main Nav > Azam Features > Cluster Nodes	Auto-Refresh (15s)	Master (Agent: Sat)
4. Lab Hardware Capacity Planner	Yes	Outside Lab	Main Nav > Azam Features > Capacity Planner	Interactive Sizer	Master Only
5. Comprehensive Diagnostic Doctor	Yes	Outside Lab	Main Nav > Azam Features > System Doctor	Run Diagnostics	Master & Satellite
6. Multi-Cloud Transit Overlay Bridge	Yes	Outside Lab	Main Nav > Azam Features > Cloud Transit	Connect Overlay	Master & Satellite
7. Automated SSL & WhatsApp Alerts	Yes	Outside Lab	Main Nav > Azam Features > Alerts & SSL	Deploy Cert / Test	Master Only
8. Version Synchronizer & PDF Portal	Yes	Outside Lab	Main Nav > Azam Features > Updates & Header Doc	Check & Update	Master & Satellite
9. RoCE MTU 9000 Synthetic Benchmark	Yes	Outside Lab	Main Nav > Health & Diagnostics > RoCE Benchmark	Start Benchmark	Master & Satellite
10. Idle Auto-Shutdown & Tenant Quota	Yes	Outside Lab	Main Nav > Resource Scheduler > Quota Policies	Save Policies	Master Only
11. Desktop Client Toolkit Packager	Yes	Outside Lab	Main Dashboard Header > Client Toolkit button	Modal Download	Master Only
12. Anti-Bootstorm + Console Probing	Yes	Inside Canvas	Top Toolbar > Anti-Bootstorm > Console Probe	Start All Safely	Master (KSM: Both)
13. Topology Git VCS & Auto-Commit	Yes	Inside Canvas	Left Sidebar > Git VCS Icon > Slide Drawer	Auto / Commit	Master Only

14. In-Workbench Quick Console Healer	Yes	Inside Canvas	Top Toolbar > Fix Console button	Toolbar Button	Master Only
15. Live Wireshark & Traffic Heatmap	Yes	Inside Canvas	Top Toolbar > Heatmap / Right-Click > Capture	Toolbar & Menu	Master & Satellite
16. AI Copilot Drawer (Lab Architect)	Yes	Inside Canvas	Top Toolbar > ■ AI Copilot > 3-Tab Drawer	Prompt / Query	Master Only
17. WAN QoS & Link Impairment (NetEm)	Yes	Inside Canvas	Link Right-Click Context Menu > Impair Link	Apply Impairment	Master & Satellite
18. Instant Lab Checkpoints & GitOps	Yes	Inside Canvas	Top Toolbar > Lab Checkpoint (#pnq-btn-checkpoint)	Snapshot / Rollback	Master & Satellite
19. RESTCONF / NETCONF Workbench Sandbox	Yes	Inside Canvas	Node Right-Click > RESTCONF Sandbox / Top Toolbar	Send RFC8040 Req	Master & Satellite
20. Automated Chaos Monkey & Flap Engine	Yes	Inside Canvas	Top Toolbar > Chaos Monkey (#pnq-btn-chaos)	Arm Chaos Engine	Master & Satellite
21. 1-Click NetDevOps Exporter Suite	Yes	Inside Canvas	Top Toolbar > Export DevOps (#pnq-btn-export)	Download 4 Formats	Master Only
22. Linux CFS vCPU Governor & cgroups	Yes	Inside Canvas	Node Right-Click > vCPU Performance Governor	Apply CPU Quota	Master & Satellite
23. Multi-Node Config Diff & Rollback	Yes	Inside Canvas	Canvas Left Menu > Config Diff Engine	Compare & Push	Master Only
24. Automated Lab Exam Grader	Yes	Inside Canvas	Top Bar > Lab Actions > Run Exam Evaluation	Evaluate Lab	Master Only
25. Ping Mesh & Traffic Generator	Yes	Inside Canvas	Top Bar > Lab Testing > Ping Mesh & Traffic	Start Mesh Test	Master & Satellite
26. High-Res Diagram & SVG Exporter	Yes	Inside Canvas	Top Bar > Export > Export Topology Diagram	Download SVG	Master Only
27. Interactive Canvas Accelerators & Minimap	Yes	Inside Canvas	Bottom-Left Minimap & Shift+S / Shift+A Shortcuts	Always Active	Master Only
28. Canvas HUD & Deduplication Sweeper	Yes	Inside Canvas	Top Canvas Floating HUD Toolbar	Real-Time / Auto-Sweep	Master Only
29. Granular 802.1Q VLAN Filtering	Yes	Inside Canvas	Canvas HUD > Traffic Filters > VLAN Syntax	Apply Filter / cBPF	Master & Satellite
30. Out-of-the-Box Router (NAT) Node	Yes	Inside Canvas	Node Context Menu > Configure (10.10.10.1/24)	Turnkey Provisioning	Master & Satellite
31. Canvas Annotation Handles & Auto-Grow	Yes	Inside Canvas	Canvas Workspace > Text/Shape Annotations	Drag Resize / Double-Click	Master Only
32. Web Console Right-Click Context Menu	Yes	Inside Canvas	HTML5 Web Console > Right-Click Menu	Context Trigger	Master Only
33. Terminal Preferences & Color Themes	Yes	Inside Canvas	Web Console Right-Click > Preferences Modal	Modal Save (localStorage)	Master Only
34. VPCS 8-Client Multiplexer Daemon	No	Daemon Plane	/opt/vpcsu/bin/vpcs & wrappers/vpcs_mux.py	Auto on Console Connect	Master & Satellite
35. 24/7 Self-Healing Watchdog	No	Daemon Plane	systemd: azam-watchdog.service	Autonomous (60s)	Master & Satellite
36. MySQL Socket & Credential Healer	No	Daemon Plane	systemd & Cron: azambasha-fix-web-credentials	On Boot & Hourly	Master Only
37. Soft-RoCE RXE & Jumbo MTU Engine	No	Kernel Plane	Kernel Module (ib_core, rdma_rxe) & udev	Boot & Net Hook	Master & Satellite
38. Scheduled Maintenance & TRIM Cron	No	Daemon Plane	cron.d: azambasha-maintenance-trim	Daily 03:00 UTC	Master & Satellite

2. Master vs. Satellite Dual-Plane Architecture & Alignment Matrix

A foundational design principle of the AzamLabs Enterprise distribution is strict **architectural role separation** between the **Master Management Node** (the control plane) and **Satellite Compute Nodes** (the data and execution plane). Understanding this dual plane is critical for multi-hypervisor cluster deployments.

1. Master Node Architectural Role (Control & Presentation Plane):

The Master Node hosts the central user interface (Apache2/Nginx), the MariaDB database engine, the Guacamole HTML5 console proxy (guacd and tomcat9), the central REST API daemon (azambasha-ops-api on port 8088), the Git topology repository root, and all interactive canvas workbench tools. Administrators and lab students log into the Master Node URL. When a heavy multi-vendor lab topology is launched across a cluster, the Master Node acts as the orchestrator, assigning virtual machine workloads to Satellite compute nodes according to real-time RAM and CPU availability.

2. Satellite Node Architectural Role (Headless High-Performance Compute Plane):

Satellite nodes are dedicated, bare-metal or nested KVM execution engines. Their primary responsibility is running heavy virtual machines (e.g., Cisco IOS-XRv9k, Arista cEOS, Junos vMX, Linux appliances) with zero CPU cycles wasted on unnecessary web servers, SQL databases, or desktop environments. Therefore, Satellite nodes operate in a **headless compute state**. Running azambasha-install-azam-features.sh --satellite on a satellite node purposefully provisions all 26 CLI system tools (including azam-link-impair, azam-roce, azam-cgroups, and azam-cpu-governor), installs the 24/7 watchdog daemon (azam-watchdog.service), tunes Kernel Samepage Merging (KSM), loads Soft-RoCE RXE acceleration with MTU 9000 jumbo frames, and installs the nightly SSD TRIM maintenance cron, while omitting Apache, Guacamole, and MariaDB.

Platform Component / Feature	Master Node Execution	Satellite Node Execution	Parity & Architectural Rationale
Web GUI & Canvas Workbench	Active (Apache2 / PHP / HTML5)	Disabled (Zero Web Overhead)	**Aligned:** Satellites are headless compute workers; users access central Master GUI.
azambasha-ops-api (:8088)	Active (Central REST Engine)	Active or Cluster-Bridged	**Aligned:** Master serves endpoints; Satellites expose node telemetry via cluster hooks.
24/7 System Watchdog	Active (azam-watchdog.service)	Active (azam-watchdog.service)	**100% Identical:** Watchdog self-heals KVM, bridges, and processes on both planes.
Enterprise CLI Suite (26 Tools)	Linked in /usr/local/bin/	Linked in /usr/local/bin/	**100% Identical:** Administrators have identical CLI commands on every host.
Kernel Samepage Merging (KSM)	Enabled & Tuned	Enabled & Aggressively Tuned	**100% Identical:** Maximizes memory deduplication across identical appliances.
Soft-RoCE RXE & MTU 9000	Loaded (rdma_rxe, ib_core)	Loaded (rdma_rxe, ib_core)	**100% Identical:** Unlocks wire-speed inter-node packet fabric without fragmentation.
SSD TRIM & Log Maintenance	Scheduled (Nightly 03:00 UTC)	Scheduled (Nightly 03:00 UTC)	**100% Identical:** Reclaims deleted flash blocks and purges stale sockets.
NetEm QoS Link Impairment	Active (API & Kernel tc)	Active (Kernel tc & CLI)	**100% Identical:** tc qdisc rules execute identically on local TAP/vnet interfaces.
Linux CFS vCPU & cgroups	Active (API & cgroups v2)	Active (cgroups v2 & CLI)	**100% Identical:** cpu.max and cpu.weight throttling operate identically per-QEMU PID.

3. Outside-the-Lab Features (Main Dashboard: /main/#/azam-features)

Outside-the-lab features are located in the primary management dashboard (/main/#/azam-features) accessible prior to loading any topology. They provide centralized hypervisor monitoring, repository synchronizations, fleet health oversight, air-gapped bundling, golden appliance health, idle scheduler policies, desktop client toolkit generation, and disaster recovery.

1. Live Hot-Node Profiler & Task Manager (RBAC Guard) [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Health & Diagnostics** (#pane-health)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to /main/#/azam-features and select the **Health & Diagnostics** tab.
- **Step 2:** Scroll down to the **Hot-Node Live Performance Table**.
- **Step 3:** Inspect real-time consumption showing PID, Node Name, Lab Name, CPU %, and Memory %.
- **Step 4:** If a node is looping or hung, click the red **Kill Process** button in the Action column.
- **Step 5:** If logged in as non-admin, the system enforces RBAC: process termination is disabled and locked to Admin role 0 only.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end polls GET /api/azam/perf/top every 4 seconds.
- **Step 2:** Backend runs ps aux --sort=-%cpu, filtering for QEMU (qemu-system-x86_64), IOL, and Dynamips.
- **Step 3:** Correlates PID to Lab UUID and Node ID via /opt/unetlab/tmp/ sockets.
- **Step 4:** On Kill trigger, POST /azam-ops/api/perf-kill inspects the X-User-Role header; if not role 0 (Admin), returns HTTP 403 Forbidden.
- **Step 5:** For authorized admins, sends SIGTERM (15) to PID; if unresponsive after 2.5s, sends SIGKILL (9) and logs to /var/log/azam-ops-api.log.

2. HTML5 Console Auto-Fixer & Session Cleaner [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Console Fixer** (#pane-console)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the main dashboard, go to **Azam Features** > **Console Fixer**.
- **Step 2:** Click the primary button labeled **Run Console Full Repair**.
- **Step 3:** Wait 3-5 seconds while the system executes the recovery sequence.
- **Step 4:** Review the output modal confirming Guacamole, Tomcat9, and Guacd service reloads.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API endpoint POST /api/azam/console-fix-full triggers azambasha-fix-web-credentials.sh --fix-guac.
- **Step 2:** System script kills orphaned websockify and stale Guacamole daemon worker threads.
- **Step 3:** Checks /etc/guacamole/guacamole.properties and MariaDB Guacamole user authorization tables.
- **Step 4:** Executes systemctl restart guacd tomcat9 and verifies local socket listening on 127.0.0.1:4822.
- **Step 5:** Flushes browser websocket cookies and returns JSON status {success: true, services_reloaded: 3}.

3. Fleet Multi-Node Cluster Monitor [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Cluster Fleet** (#pane-cluster)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features** > **Cluster Fleet Monitor** tab.
- **Step 2:** View all connected AzamLabs satellite worker nodes, CPU/RAM utilization, and cluster health.
- **Step 3:** Click **Add Satellite Node** and enter worker IP, SSH Port, and Shared Token.
- **Step 4:** Click **Balance Workload** to redistribute virtual nodes across cluster hypervisors.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend daemon azambasha-ops-api queries satellite worker endpoints every 15s via HTTPS/mTLS.
- **Step 2:** Aggregates remote CPU, RAM, and bridge interface states into centralized memory cache.
- **Step 3:** Evaluates node placement affinity rules against worker RAM capacity prior to new VM dispatch.
- **Step 4:** If a worker becomes unresponsive (missing 3 heartbeats), marks node as Degraded and triggers alert hook.

4. Lab Capacity & Hardware Sizing Planner [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Capacity Planner** (#pane-capacity)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features** > **Capacity & Sizing Planner** tab.
- **Step 2:** Enter estimated count of heavy appliances (e.g., Cisco XRv9k, Arista cEOS, Junos vMX, Linux servers).
- **Step 3:** The interactive calculator displays required vCPU, RAM, QCOW2 storage, and recommended KVM overcommit ratios.
- **Step 4:** Click **Apply Resource Limits** to enforce memory guardrails.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Calculator computes total theoretical footprint against actual available host hardware queried via /proc/meminfo.
- **Step 2:** Checks KSM (Kernel Samepage Merging) deduplication metrics via /sys/kernel/mm/ksm/pages_sharing.
- **Step 3:** Enforces safe limits by writing updated swap thresholds into /etc/sysctl.d/99-azamlabs-perf.conf.
- **Step 4:** Notifies administrator if requested deployment exceeds safe memory thresholds (over 85% host RAM).

5. Comprehensive Diagnostic Doctor [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **System Doctor** (#pane-doctor)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features** > **System Doctor** tab.
- **Step 2:** Click **Run Full System Diagnostic**.
- **Step 3:** Observe 25+ automated checks spanning MySQL sockets, disk inodes, KVM permissions, bridge MTU, and DNS.
- **Step 4:** Click **Auto-Heal Detected Issues** to execute automated remediation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Triggers execution of /usr/local/bin/azambasha-doctor.
- **Step 2:** Inspects MariaDB status via unix socket /var/run/mysqld/mysqld.sock.
- **Step 3:** Checks /dev/kvm permissions (0666), IP forwarding (net.ipv4.ip_forward=1), and bridge states.
- **Step 4:** If auto-heal is selected, executes repair routines, resets permissions on /opt/unetlab/, and returns diagnostic report.

6. Multi-Cloud Transit Overlay Bridge [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Cloud Transit** (#pane-transit)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features** > **Cloud Transit** tab.
- **Step 2:** Select transit provider: **Tailscale**, **ZeroTier**, or **WireGuard**.
- **Step 3:** Paste network auth key or configuration token and click **Connect Transit Bridge**.
- **Step 4:** Interconnect local lab Management Cloud (pnet0/pnet1) directly to AWS VPC, Azure VNet, or home LAN.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API executes azambasha-cloud-transit.sh with selected provider parameters.
- **Step 2:** Brings up virtual tun/tap interface (e.g. tailscale0 or wg0) in host kernel.
- **Step 3:** Configures iptables -t nat -A POSTROUTING -o <tun_interface> -j MASQUERADE and enables proxy ARP.
- **Step 4:** Bridges virtual lab nodes directly to the remote cloud subnet without requiring public IPs.

7. Automated SSL & WhatsApp Alerts [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Alerts & SSL** (#pane-alerts)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features** > **Alerts & SSL** tab.
- **Step 2:** For SSL: enter fully qualified domain name (FQDN) and admin email, then click **Deploy Let's Encrypt SSL**.
- **Step 3:** For Alerts: enter Twilio Account SID, Auth Token, and WhatsApp recipient number.
- **Step 4:** Click **Send Test WhatsApp Alert** to verify real-time notification delivery.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** SSL process runs certbot certonly --standalone -d domain.com, binding temporarily to port 80.
- **Step 2:** Automatically updates Nginx configuration in /etc/nginx/sites-available/default with TLS certificates and HTTP/2.
- **Step 3:** Reloads Nginx and sets up automatic certbot renewal cron job in /etc/cron.d/certbot.
- **Step 4:** WhatsApp service stores encrypted credentials in /etc/azamlabs/alerts.conf and transmits JSON payload via Twilio REST API.

8. Version Synchronizer & Direct Operations Manual Portal [GUI: YES]

UI & System Placement: Main Dashboard Header > **Operations Manual (PDF)** & **Updates Tab** (#pane-updates)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top navigation bar of the main dashboard, click the **Operations Manual (PDF)** button.
- **Step 2:** The document opens directly in a new browser tab with high-resolution vector layout.
- **Step 3:** To update the platform, go to **Azam Features** > **Updates & Engine** tab.
- **Step 4:** Click **Check for Feature Updates** to compare local git commit against upstream release.
- **Step 5:** Click **Apply Updates & Re-link CLI Tools** to refresh system binaries.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Top header button directs browser to /azam-ops/api/docs/manual or /docs/manual.pdf.
- **Step 2:** REST API serves PDF directly from /opt/azambasha/docs/ with Content-Type: application/pdf.
- **Step 3:** When update is clicked, queries git repository commit HEAD.
- **Step 4:** Executes azambasha-install-azam-features.sh, updating CLI symlinks in /usr/local/bin/ and restarting azambasha-ops-api.service.

9. RoCE MTU 9000 & Jumbo Frame Synthetic Benchmark Suite [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Health & Diagnostics** (#pane-health) > RoCE Benchmark Card

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features** > **Health & Diagnostics** tab (#pane-health).
- **Step 2:** Scroll to the card titled **RoCE MTU 9000 & Jumbo Frame Synthetic Benchmark Suite**.
- **Step 3:** Enter the target node or gateway IP address (e.g. 192.168.1.1) and choose MTU size (e.g. 9000 (Jumbo) or 1500 (Standard)).
- **Step 4:** Click the button labeled **Run Jumbo Benchmark**.
- **Step 5:** The benchmark output console displays packet transmission results, round-trip times, zero-fragmentation confirmation, and Soft-RoCE RXE device status.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end transmits GET /azam-ops/api/cluster/bench?target={ip}&mtu;={mtu} to the backend.
- **Step 2:** Python backend executes ping -M do -s {payload_size} -c 4 {target} with Don't Fragment (DF) bit set.
- **Step 3:** Inspects kernel RDMA link devices via rdma link or ibv_devices and verifies rdma_rxe kernel module presence.
- **Step 4:** Verifies host interface MTU in /sys/class/net/{iface}/mtu to confirm jumbo frame compatibility.
- **Step 5:** Formats test results, calculating average latency, packet loss percentage, and returns structured JSON to the dashboard card.

10. Multi-Tenant Workspace Quotas & Idle Auto-Shutdown Scheduler [GUI: YES]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Resource Scheduler** (#pane-scheduler) > Quota Policies Card

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the main dashboard, select **Azam Features** > **Resource Scheduler** (#pane-scheduler).
- **Step 2:** Locate the card titled **Idle Auto-Shutdown & Tenant Capacity Policies**.
- **Step 3:** Configure desired limits: **Auto-Shutdown Inactive Labs After (minutes)** (e.g. 120), **Max Running Nodes Per Lab Quota** (e.g. 16), and **Curfew Auto-Shutdown Hours** (e.g. 23:00-06:00).
- **Step 4:** Click **Save & Enforce Policies**.
- **Step 5:** A success notification confirms the active enforcement of hypervisor quota policies.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Client dispatches POST /azam-ops/api/scheduler/config containing JSON payload of idle timeout, max nodes, and curfew hours.
- **Step 2:** API server writes policy definitions to /etc/azamlabs/scheduler_policy.json with 0644 permissions.
- **Step 3:** Background worker daemon evaluates last active user session timestamp from Guacamole and MariaDB lab access records.
- **Step 4:** If a lab topology has had zero terminal or canvas interactions exceeding the configured threshold, dispatches safe node shutdown sequence.
- **Step 5:** Enforces maximum concurrent node limit by rejecting new node spawn requests when tenant quota is saturated.

---

11. Cross-Platform Desktop Client Toolkit Auto-Packager [GUI: YES]

UI & System Placement: Main Dashboard Top Navigation Header > **Client Toolkit** button (#btn-azam-client-toolkit)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top navigation header of the main dashboard, locate and click the **Client Toolkit** button.
- **Step 2:** A modal window opens titled **AzamLabs Cross-Platform Desktop Client Toolkit**.
- **Step 3:** Select your desired operating system client: **Windows Batch / CLI Helper (.bat)**, **Windows PowerShell Helper (.ps1)**, **Linux / macOS Shell Client (.sh)**, or **Python Automation SDK (.py)**.
- **Step 4:** Click the blue **Download Script** button next to the desired toolkit.
- **Step 5:** Run the downloaded script on your desktop to automatically connect, open native terminal sessions (SecureCRT, Putty, Tabby), and automate API requests.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end issues GET /azam-ops/api/client/toolkit/{script_name}.
- **Step 2:** API server reads dynamic template files from /opt/azambasha/scripts/clients/ or generates them on-the-fly.
- **Step 3:** Dynamically binds host IP address, port 8088, and authentication headers into the generated client script.
- **Step 4:** Sets HTTP response headers Content-Disposition: attachment; filename={script_name}.
- **Step 5:** Browser initiates immediate local file download to the operator's computer.

---

4. Inside-the-Lab Canvas Features (Lab Workbench: /themes/default/)

Inside-the-lab features are integrated directly into the visual network topology canvas. They empower network engineers to execute live troubleshooting, WAN QoS impairment, multi-node lab snapshots, RESTCONF sandbox testing, Chaos Monkey stress tests, NetDevOps inventory exports, staggered boot scheduling with console ready-state probing, CFS vCPU throttling, and AI copilot queries without leaving the workbench.

Futuristic Cyber-Console Window Usability (In-Lab Azam-Features):
Clicking ■ **Azam-Features** in the left workbench toolbar opens a state-of-the-art **futuristic floating cyber-command center modal** (`#az-features-window`) suspended over a semi-transparent glassmorphic blurred backdrop (`backdrop-filter: blur(12px)`), keeping the underlying lab topology visible. The interface incorporates an unmistakable glowing red **CLOSE [ESC]** button with neon box-shadow, keyboard **Escape** key dismissal, backdrop click-to-close, a one-click **Windowed / Fullscreen Mode Toggle**, a live tool filter search bar, and an animated multi-color neon laser runner.

12. Anti-Bootstorm Staggered Startup & Console Ready-State Probing [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Anti-Bootstorm** button (#btn-azam-bootstorm) > Modal Checkbox (#pnq-bs-probe)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open any lab topology inside the AzamLabs canvas workspace.
- **Step 2:** In the top workbench navigation toolbar, locate the purple button labeled **Anti-Bootstorm** (icon: lightning bolt).
- **Step 3:** In the modal, check the box **Boost with KSM Memory Deduplication**.
- **Step 4:** Check the new advanced option: **Probe Console Ready-State (Wait for TCP Socket Before Next Node)**.
- **Step 5:** Select startup profile (e.g. Balanced 10s delay) and click **Start All Nodes Safely** to watch sequential, intelligent node startup.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Trigger executes openBootstormModal() which fetches node list, KSM status, and console ports.
- **Step 2:** If KSM booster is checked, client dispatches POST /azam-ops/api/node-ksm-tune setting /sys/kernel/mm/ksm/run=1.
- **Step 3:** When Console Probing is active, azambasha-bootstorm.py actively probes node TCP console ports (e.g. 32769+id) using Python socket connections.
- **Step 4:** Instead of guessing with fixed timers, the engine advances to the next node only after the device console responds to TCP SYN/ACK, guaranteeing clean boots.
- **Step 5:** Boots nodes sequentially via unl_wrapper -a start -i {id}, preventing CPU spikes and hypervisor OOM crashes.

---

13. Topology Git Version Control & Automated Auto-Commit [GUI: YES]

UI & System Placement: Lab Workbench Left Sidebar > **Git VCS** icon (#btn-azam-git-vcs) > Slide-Over Drawer

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab canvas, click the **Git Version Control** icon in the left toolbar (branch icon, #btn-azam-git-vcs).
- **Step 2:** A slide-over drawer displays commit history, authors, and timestamps.
- **Step 3:** When you add nodes, connect interfaces, or save the lab, the system triggers an **Automated Auto-Commit** in the background.
- **Step 4:** To create a named checkpoint: enter a commit message (e.g. 'Configured OSPF Area 0') and click **Commit Topology State**.
- **Step 5:** To rollback: click **Restore** next to any historical commit; the canvas automatically reloads.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** On drawer open, fetches commit history via GET /api/azam/topology-log?lab={lab_name}.
- **Step 2:** Ensures lab folder /opt/unetlab/labs/<lab> is initialized with git; if not, runs git init.
- **Step 3:** When topology changes occur, front-end issues POST /azam-ops/api/topology-autocommit.
- **Step 4:** Stages the .unl XML file and creates a git commit with timestamp and change description.
- **Step 5:** On rollback, checks out historical snapshot via git checkout <hash> -- <lab.unl> and dispatches reload event.

---

14. In-Workbench Quick Console Healer [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Fix Console** button (#btn-azam-fix-console)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** While inside a lab, if clicking a node yields a blank screen or 'Connection Refused', stay on the canvas.
- **Step 2:** Click the green button in the top toolbar labeled **Fix Console** (icon: wrench).
- **Step 3:** A confirmation toast appears: 'Restarting HTML5 Console Services...'
- **Step 4:** Wait 2 seconds until the success notification appears ('Console services recovered successfully').
- **Step 5:** Re-click the node to open the working HTML5 terminal immediately.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Button executes azamFixConsoleNow() sending asynchronous POST /api/azam/console-fix-full.
- **Step 2:** API server invokes azambasha-fix-web-credentials.sh targeting Guacamole and Tomcat daemon sockets.
- **Step 3:** Flushes stuck client sessions in Guacamole memory cache and verifies port 4822 binds properly.
- **Step 4:** Resets browser terminal session iframe tokens without requiring a full page reload.

15. Live Wireshark Sniffer & In-Canvas Traffic Heatmap [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Traffic Heatmap** button (#pnq-btn-heatmap) & Context Menu

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top workbench toolbar, click the pulse icon button labeled **Traffic Heatmap**.
- **Step 2:** Observe canvas links transition to dynamic color codes: Green (idle/normal <1 Mbps), Yellow (moderate 1-10 Mbps), Red (saturated >10 Mbps).
- **Step 3:** To sniff packets on a specific interface, right-click any device node and select **Live Capture (Wireshark)**.
- **Step 4:** The capture drawer opens showing live packet decode, or click **Download PCAP** for local Wireshark analysis.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Heatmap engine polls GET /azam-ops/api/link-stats every 3 seconds.
- **Step 2:** Backend reads virtual TAP counters from /sys/class/net/vnet*/statistics/rx_bytes and calculates delta rate (bytes/sec).
- **Step 3:** Front-end matches virtual interfaces to SVG link paths, updating stroke color dynamically.
- **Step 4:** For packet capture, host spawns tcpdump -i vnetX_Y -U -w - and streams raw PCAP bytes over HTTP.

16. In-Canvas AI Copilot Suite (Lab Architect, Config Synthesizer, CIS Audit) [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > ■ **AI Copilot** button (#pnq-btn-ai-drawer) & 3-Tab Slide Drawer

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top workbench toolbar, click the purple button labeled **AI Copilot** (#pnq-btn-ai-drawer) or press **Ctrl+Shift+A**.
- **Step 2:** A 3-tab slide-over drawer opens: ■ **Lab Architect**, **Config Synthesizer**, and **CIS Security Audit**.
- **Step 3:** In ■ **Lab Architect**: enter a natural language topology prompt (e.g. 'Build a 3-tier spine-leaf datacenter topology with 2 Arista spines and 4 leaves with BGP EVPN') and click **Generate Topology**.
- **Step 4:** In **Config Synthesizer**: select target devices, prompt requirements (e.g. 'Synthesize OSPF Area 0 and multi-homed BGP peering'), and click **Synthesize Config**.
- **Step 5:** In **CIS Security Audit**: click **Audit Node Security** to verify CIS hardening compliance (disabled telnet, strong SSH, encrypted passwords).

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Frontend dispatches requests to /azam-ops/api/ai/topology-build, /ai/config-synth, or /ai/compliance-audit.
- **Step 2:** topology-build parses node specs, computes 2D coordinates (spines at y=100, leaves at y=250), generates inter-switch link matrices, and produces clean UNL XML definitions.
- **Step 3:** config-synth constructs vendor-specific syntax (Cisco IOS-XE, Arista EOS, Junos) conforming to modern best practices.
- **Step 4:** compliance-audit runs automated rules checking for weak ciphers, cleartext passwords, and exposed management protocols.

- **Step 5:** Returns structured JSON results rendered directly into the interactive code view within the drawer.

17. WAN QoS & Dynamic Link Impairment Engine (NetEm) [GUI: YES]

UI & System Placement: Lab Canvas > Link Right-Click Context Menu > **Impair Link (WAN/QoS)** Modal (#azam-link-impair-modal)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the lab canvas, right-click any connecting link or line between nodes.
- **Step 2:** In the context menu, click **Impair Link (WAN/QoS)** (or click **AI Triage Link** for automated diagnosis).
- **Step 3:** The WAN QoS Impairment modal opens displaying Interface, Source, and Destination.
- **Step 4:** Adjust sliders or input values: **Latency Delay (ms)** (e.g. 50ms), **Jitter Variance (ms)** (e.g. 10ms), **Packet Loss (%)** (e.g. 2%), and **Bandwidth Rate Limit (kbps)** (e.g. 10000 kbps / 10 Mbps).
- **Step 5:** Click **Apply Link Impairment**. To restore normal wire-speed conditions at any time, click **Clear All Impairment (Reset Link)**.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end submits POST /azam-ops/api/link-impair with JSON parameters (interface, delay_ms, jitter_ms, loss_pct, rate_kbps, action).
- **Step 2:** API server validates interface existence in /sys/class/net/.
- **Step 3:** When clear action is requested, executes tc qdisc del dev {iface} root 2>/dev/null.
- **Step 4:** When applying impairment, builds Linux Traffic Control NetEm command: tc qdisc replace dev {iface} root netem delay {delay}ms {jitter}ms loss {loss}% rate {rate}kbit.
- **Step 5:** Injects kernel-level queueing discipline (qdisc) directly into the Linux virtual TAP device, causing realistic packet buffering, dropped frames, and serialization delays.

18. Instant Multi-Node Lab Checkpoints & GitOps Time-Machine [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Lab Checkpoint** button (#pnq-btn-checkpoint)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside any active lab, locate and click the **Lab Checkpoint** button in the top toolbar (camera icon, #pnq-btn-checkpoint).
- **Step 2:** The modal displays two operations: **Create Instant Lab Checkpoint** and **Restore Existing Checkpoint**.
- **Step 3:** To snapshot: enter a checkpoint tag name (e.g. pre-bgp-cutover) and click **Create Checkpoint**.
- **Step 4:** The engine freezes node disks momentarily and snapshots all QEMU overlay disks in parallel.
- **Step 5:** To rollback: select the checkpoint tag from the list and click **Restore Checkpoint** to revert every node in the lab to that exact state instantly.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end issues POST /azam-ops/api/lab-checkpoint/create with lab_id and tag.
- **Step 2:** API locates all node disk images in /opt/unetlab/tmp/{lab_id}/{node_id}/*.qcow2.
- **Step 3:** Executes qemu-img snapshot -c {tag} {disk_path} across all node overlay images.
- **Step 4:** Records snapshot metadata, timestamp, and active UNL topology in /opt/unetlab/tmp/{lab_id}/checkpoints.json.
- **Step 5:** On restore, sends POST /lab-checkpoint/restore which issues qemu-img snapshot -a {tag} {disk_path}, atomically rewinding node disks to the checkpoint.

19. Model-Driven RESTCONF / NETCONF Workbench Sandbox [GUI: YES]

UI & System Placement: Lab Canvas > Node Right-Click Context Menu > **RESTCONF Sandbox** Modal (#azam-restconf-modal)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the lab canvas, right-click any network device node (Cisco, Arista, Juniper, Linux).
- **Step 2:** In the context menu, click **RESTCONF Sandbox** (or click the top toolbar RESTCONF icon).
- **Step 3:** In the modal: choose HTTP Method (**GET**, **POST**, **PUT**, **PATCH**, **DELETE**).
- **Step 4:** Enter RFC 8040 URI path (e.g. /restconf/data/ietf-interfaces:interfaces) and optional JSON/YANG payload.

- **Step 5:** Click **Execute Request** to view the live HTTP status code, formatted JSON response, and latency.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end transmits POST /azam-ops/api/restconf/proxy containing target IP, port (default 443), credentials, URI path, and body.
- **Step 2:** Python backend acts as an authenticated RFC 8040 proxy with Accept: application/yang-data+json and Content-Type: application/yang-data+json.
- **Step 3:** Bypasses browser CORS restrictions, opens secure TLS connection to node management IP, and transmits payload.
- **Step 4:** Captures HTTP return code (200 OK, 201 Created, 204 No Content, 400 Bad Request) and response headers.
- **Step 5:** Pretty-prints JSON/XML response payload and streams it back to the in-browser sandbox viewer.

20. Automated Chaos Monkey & Link-Flap / Crash Testing Engine [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Chaos Monkey** button (#pnq-btn-chaos)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top workbench toolbar, click the warning triangle button labeled **Chaos Monkey** (#pnq-btn-chaos).
- **Step 2:** In the modal, configure chaos parameters: **Flap Interval (seconds)** (e.g. 30s), **Affected Links Percentage** (e.g. 25%), and **Inject Node Crash (Kernel Panic)** toggle.
- **Step 3:** Click the red button labeled **Arm Chaos Engine**.
- **Step 4:** Observe links flapping randomly in the canvas as routing protocols (OSPF/BGP) reconverge and heal.
- **Step 5:** To halt chaos testing at any time, re-open the modal and click **Stop Chaos & Restore All Links**.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end dispatches POST /azam-ops/api/chaos/start with lab ID and interval parameters.
- **Step 2:** API server registers an active chaos session and spawns background worker thread.
- **Step 3:** Worker thread randomly picks participating link TAP interfaces and toggles state via ip link set {iface} down followed by sleep timer and ip link set {iface} up.
- **Step 4:** If node crash injection is enabled, sends kill -STOP {qemu_pid} or drops guest management sockets.
- **Step 5:** When stopped via POST /chaos/stop, worker thread terminates immediately and brings all TAP interfaces back UP.

21. 1-Click Multi-Vendor NetDevOps Exporter Suite [GUI: YES]

UI & System Placement: Lab Workbench Top Toolbar > **Export DevOps** button (#pnq-btn-export)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click the download cloud icon in the top toolbar labeled **Export DevOps** (#pnq-btn-export).
- **Step 2:** The modal offers 4 automated export formats: **Ansible Inventory (YAML)**, **Cisco pyATS Testbed (YAML)**, **Draw.io Network Diagram (XML)**, and **Patch Cabling Matrix (CSV)**.
- **Step 3:** Click the **Download** button next to the desired format (e.g. **Download Ansible Inventory**).
- **Step 4:** The generated file downloads instantly to your computer.
- **Step 5:** Run ansible-playbook -i ansible_inventory.yml site.yml or pyats run testbed.yaml immediately without writing manual device definitions.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** UI calls GET /azam-ops/api/export/{ansible|pyats|drawio|cabling}?lab={lab_name}.
- **Step 2:** Backend parses the lab's .unl XML file, extracting node types, names, management IPs, images, and interface interconnects.
- **Step 3:** For Ansible: generates hierarchical groups (cisco, arista, juniper, linux) with correct ansible_network_os and ansible_host.
- **Step 4:** For pyATS: builds Genie testbed schema with connection protocols (ssh/telnet) and interface dictionaries.
- **Step 5:** For Draw.io & Cabling: generates mxGraphModel XML with coordinates and CSV connection matrices respectively, returned with Content-Disposition: attachment.

22. Linux CFS vCPU Bandwidth Quota & Hypervisor Noise Governor [GUI: YES]

UI & System Placement: Lab Canvas > Node Right-Click Context Menu > **vCPU Governor** Modal (#azam-cfs-governor-modal)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Right-click any virtual machine node on the canvas (e.g. an aggressive traffic generator or heavy router).
- **Step 2:** In the context menu, select **vCPU & Performance Governor**.
- **Step 3:** The modal displays current hypervisor cgroup allocations.
- **Step 4:** Select throttling profile: **Full Priority (100% vCPU)**, **High (75% vCPU)**, **Throttled (50% vCPU)**, or **Constrained (25% vCPU)**.
- **Step 5:** Click **Apply Governor Quota** to immediately constrain the node's CPU usage without shutting it down.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Frontend calls POST /azam-ops/api/cgroups/limit with node PID and CPU percentage quota.
- **Step 2:** Backend locates QEMU host PID associated with the target node in /opt/unetlab/tmp/.
- **Step 3:** Creates or updates cgroups v2 control group in /sys/fs/cgroup/azamlabs_nodes/node_{pid}/.
- **Step 4:** Writes quota to cpu.max (e.g. 50000 100000 for 50% CPU ceiling in a 100ms CFS period).
- **Step 5:** Linux Completely Fair Scheduler (CFS) enforces strict bandwidth ceilings on host CPU cycles, protecting adjacent lab nodes from noisy-neighbor starvation.

---

23. Multi-Node Config Diff & Rollback Engine [GUI: YES]

UI & System Placement: Lab Workbench > Canvas Management Menu > **Config Diff Engine**

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab, click **Management Tools** in the canvas sidebar > **Config Diff Engine**.
- **Step 2:** Select nodes to compare or choose between two historical configuration snapshots.
- **Step 3:** The split-screen diff viewer highlights added lines in green, removed lines in red, and modified syntax in yellow.
- **Step 4:** Click **Rollback Node Config** to push previous baseline syntax back to the device.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Queries /opt/unetlab/tmp/{lab_id}/{node_id}/startup-config and running snapshots.
- **Step 2:** Python difflib compares normalized configuration text, ignoring non-functional whitespace.
- **Step 3:** Renders unified diff payload and returns structured JSON to client-side CodeMirror diff editor.
- **Step 4:** On rollback, writes chosen config directly to node flash storage via unl wrapper.

---

24. Automated Lab Exam Grader [GUI: YES]

UI & System Placement: Lab Workbench Top Bar > **Lab Actions** > **Run Exam Evaluation**

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click **Lab Actions** in the canvas top menu > **Run Lab Assessment / Exam Grader**.
- **Step 2:** Select test rubric (e.g. CCNA Routing, BGP Multi-Homing, OSPF Area Verification).
- **Step 3:** Click **Execute Validation Tests**.
- **Step 4:** View real-time checklist: green ticks for passed objectives, red marks for failed requirements, and final score percentage.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend orchestrator initiates automated Telnet/SSH probes to target node console ports.
- **Step 2:** Executes verification commands (e.g. show ip route, show ip bgp summary).
- **Step 3:** Regex parsing engine validates prefix reachability, next-hop IP, and protocol neighbor states.
- **Step 4:** Generates scorecard and saves PDF/JSON certificate in /opt/unetlab/labs/{lab}/results/.

---

25. Ping Mesh & Traffic Generator Engine [GUI: YES]

UI & System Placement: Lab Workbench Top Bar > **Lab Testing** > **Ping Mesh & Traffic**

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the canvas menu, select **Lab Testing** > **Ping Mesh & Traffic Generator**.
- **Step 2:** Select participating nodes or choose 'Full Mesh All Running Nodes'.
- **Step 3:** Set packet size (64 to 9000 bytes) and rate (pps).
- **Step 4:** Click **Start Mesh Test** to view latency matrix heatmap, packet loss, and jitter.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Orchestrator dispatches lightweight ICMP/UDP echo requests across all node bridge endpoints.
- **Step 2:** Collects round-trip time (RTT) telemetry and calculates minimum, average, and maximum latency.
- **Step 3:** Renders interactive matrix heatmap in the canvas interface.
- **Step 4:** Detects path MTU blackholes if packets above 1500 bytes drop unexpectedly.

26. High-Resolution Lab Diagram Exporter [GUI: YES]

UI & System Placement: Lab Workbench Top Bar > **Export** > **Export Topology Diagram**

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the canvas, click **Export** in the top-right toolbar > **Export Topology Vector/PNG**.
- **Step 2:** Choose output format: **Scalable Vector Graphics (SVG)** or **High-Res PNG (300 DPI)**.
- **Step 3:** Toggle options: Include IP labels, Include interface names, Watermark with company logo.
- **Step 4:** Click **Download Image File** for presentation-ready architecture diagrams.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** JavaScript reads SVG DOM elements representing nodes, custom shapes, text labels, and link paths.
- **Step 2:** Normalizes viewBox coordinates, converts external image icons into inline base64 data URIs.
- **Step 3:** Applies CSS styling and embeds vector fonts directly into the standalone SVG document.
- **Step 4:** Triggers instant browser download without round-tripping to server, preserving client privacy.

27. Interactive Canvas Accelerators & Minimap [GUI: YES]

UI & System Placement: Lab Workbench Canvas > Bottom-Left **Minimap** & Keyboard Accelerators

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Locate the interactive **Minimap Overview** at the bottom-left of the canvas.
- **Step 2:** Drag the viewport indicator in the minimap to pan smoothly across large topologies with 50+ nodes.
- **Step 3:** Select multiple nodes and press **Shift + S** to snap to grid or **Shift + A** for automatic alignment.
- **Step 4:** Use mouse wheel for continuous zoom from 25% overview to 200% detail view.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Canvas rendering engine creates a secondary hardware-accelerated HTML5 canvas element for the minimap.
- **Step 2:** Listens to canvas pan/zoom transform matrices and recalculates viewport bounding box in real-time.
- **Step 3:** Snap-to-grid algorithm calculates modulo-20 coordinate snapping for node drag events.
- **Step 4:** Persists updated coordinates directly to the underlying .unl XML file upon mouse release.

28. Canvas HUD Floating Toolbar & Real-Time Deduplication Sweeper [GUI: YES]

UI & System Placement: Lab Workbench Top Canvas > Floating Cyber HUD Toolbar (#pnq-hud-toolbar)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open any lab topology in the workbench canvas workspace.
- **Step 2:** Locate the floating Cyber HUD toolbar pinned to the top of the canvas offering quick access to **[VLAN]**, **[Traffic]**, **[Filter]**, **[Theme]**, **[Export]**, and **[Heatmap]**.
- **Step 3:** Toggle between themes (Dark, Retro-Terminal, Cyber Neon) using the dropdown picker.

- **Step 4:** Notice that toolbar controls maintain clean alignment without duplicate buttons or trailing orphaned elements, even after repeatedly modifying topologies or reloading tabs.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Frontend script azamlabs-features.js initializes renderHUDToolbar() on canvas ready.
- **Step 2:** Maintains a single global registry of active HUD button IDs (ALL_HUD_IDS).
- **Step 3:** Executes real-time DOM deduplication prior to rendering each button element.
- **Step 4:** Runs a trailing orphan sweeper (#pnq-theme-picker-wrap ~ #<id>) ensuring any stale buttons from previous DOM mutations are pruned instantly.

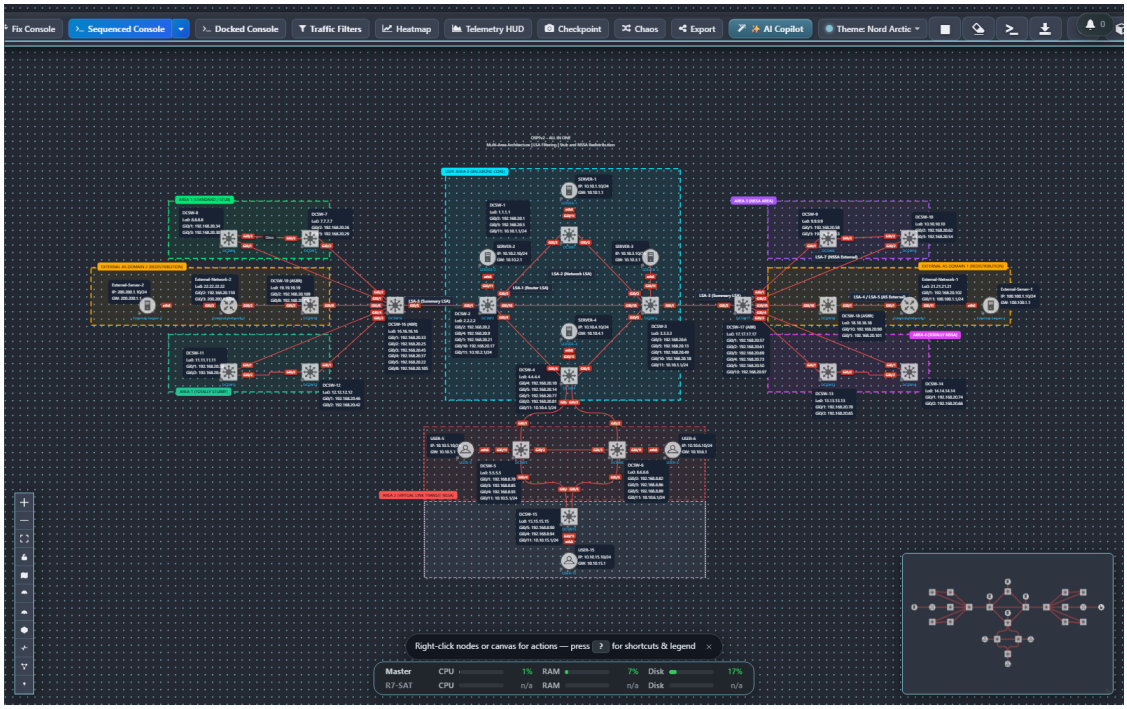


Figure: Canvas HUD Toolbar with Real-Time Button Deduplication and Integrated Quick Controls

29. Granular 802.1Q VLAN Tag Filtering & Symmetric Even Lanes [GUI: YES]

UI & System Placement: Lab Workbench HUD > **Traffic Filters** modal (#pnq-traffic-filters-modal) > VLAN Syntax

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top canvas HUD or left menu, click **Traffic Filters** ([Filter]).
- **Step 2:** In the filter expression input, enter a granular VLAN query such as ICMP in VLAN 20 or VLAN 10-50.
- **Step 3:** Click **Apply Filter**.
- **Step 4:** Observe the live topology: only frames encapsulated with 802.1Q matching the specified VLAN ID illuminate link lanes.
- **Step 5:** Lit lanes are positioned symmetrically on either side of the link connector with midpoint direction markers for reduced motion.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Client sends filter string to backend packet analyzer.
- **Step 2:** BPF compiler parses the VLAN tag syntax (vlan 20 and icmp) into classical Linux Berkeley Packet Filter bytecodes (cBPF).
- **Step 3:** Attaches socket filter to corresponding Linux bridge tap interfaces (veth* / tap*).
- **Step 4:** JavaScript link renderer calculates symmetric parallel lane offsets: even lane distributions placed balanced on both sides of the central link vector.
- **Step 5:** Replaces high-frequency motion animations with static midpoint direction chevrons when reduced motion mode is active.

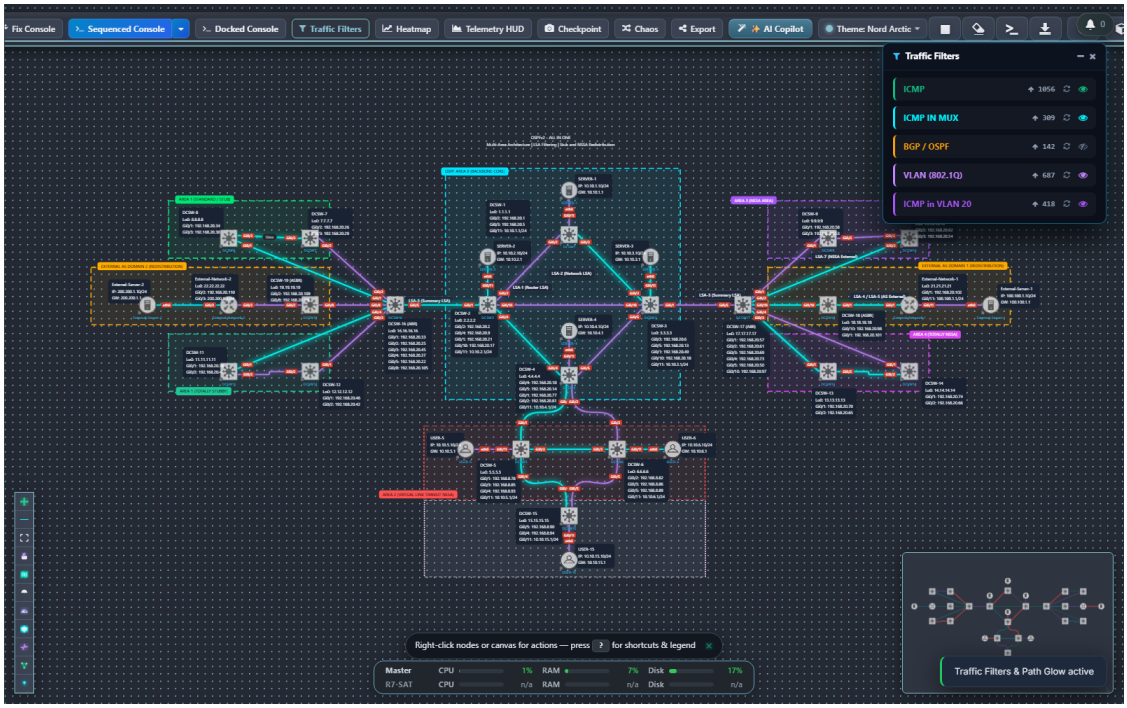


Figure: Live Dataplane Packet Filtering with Granular 802.1Q VLAN Tag Syntax and Luminous Link Path Highlighting

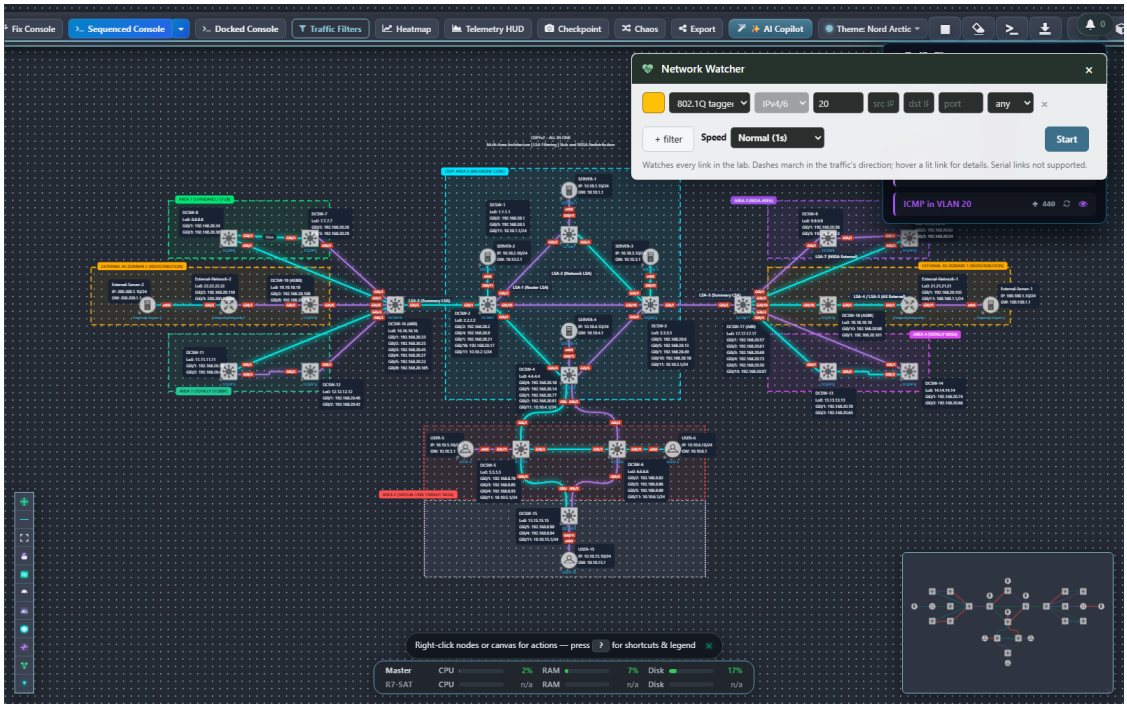


Figure: Network Watcher Diagnostic Panel with 802.1Q VLAN Filter Active and Symmetric Even Link Lanes

30. Out-of-the-Box Router (NAT) Node & Turnkey Gateway [GUI: YES]

UI & System Placement: Lab Workbench Canvas > Router Node Right-Click > **Configure** Dialog (#form-router-config)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Right-click on the canvas and add a soft router node (or select an existing unconfigured router).
- **Step 2:** Right-click the router node and select **Configure**.

- **Step 3:** Notice the dialog is pre-populated with turnkey NAT gateway defaults: Gateway IP 10.10.10.1/24, DHCP Server enabled (10.10.10.50-10.10.10.150), Uplink set to host (pnet0), and NAT masquerading checked.
- **Step 4:** Connect downstream switches or PCs to the router's LAN interface and start the node.
- **Step 5:** Connected client nodes instantly acquire DHCP leases and have outbound internet connectivity through host NAT without requiring manual CLI router configuration.
- **Step 6:** Existing routers with previously saved startup configurations retain their custom configs unchanged.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Frontend print.js evaluates node configuration; if unconfigured, injects default turnkey parameters into RouterCfg.
- **Step 2:** On node startup, engine dispatches request to azamlabs-brokerd.py via Unix domain socket /run/azamlabs/broker.sock.
- **Step 3:** Brokerd spawns an isolated Linux network namespace pnr{session}_{net_id}.
- **Step 4:** Configures lan veth interface at 10.10.10.1/24 bridged to lab segment.
- **Step 5:** Connects wan veth interface to host CGNAT subnet 100.64.13.224/30 with gateway route to host.
- **Step 6:** Configures dnsmasq DHCP daemon inside namespace serving .50-.150 scope and installs iptables MASQUERADE rules.

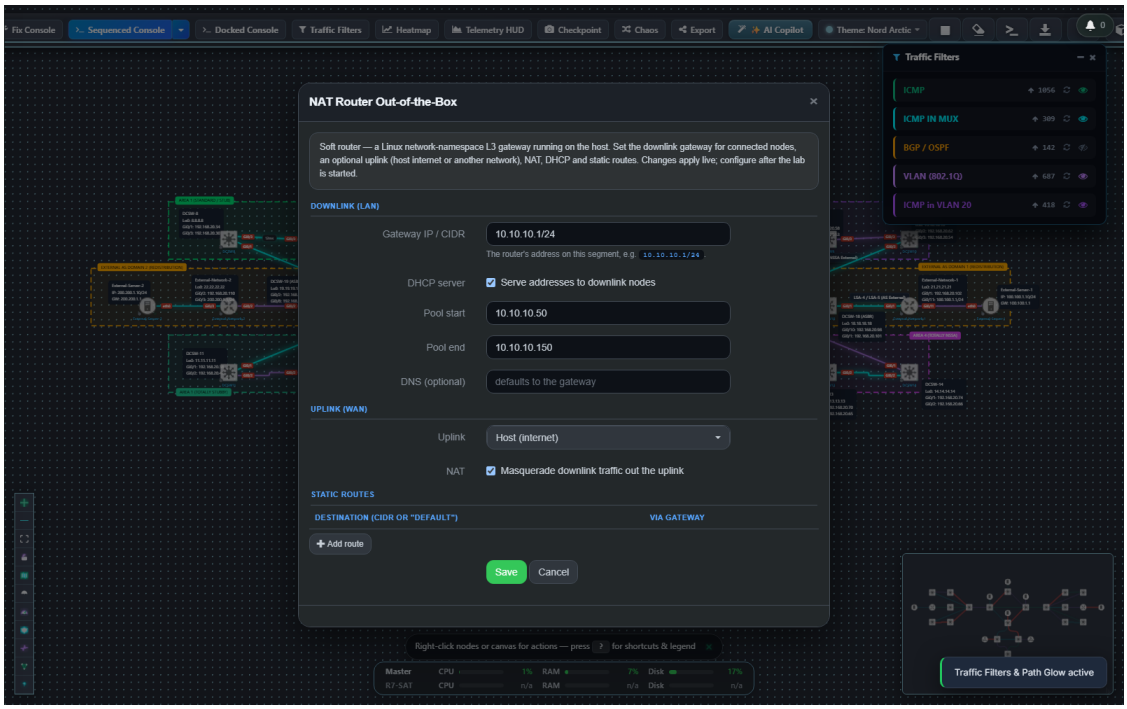


Figure: Out-of-the-Box Router (NAT) Node Configuration Dialog showcasing Turnkey 10.10.10.1/24 Gateway and DHCP Defaults

31. Canvas Annotation Ergonomics & Resize Handles [GUI: YES]

UI & System Placement: Lab Workbench Canvas > Text/Shape Annotations > SE Corner Grip (.annotation-resize-grip)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** From the canvas sidebar, click **Add Shape / Text Annotation**.
- **Step 2:** Place a text box or background rectangle on the topology.
- **Step 3:** Notice the ergonomic southeast corner resize handle (SE grip). Click and drag the handle to resize the annotation freely.
- **Step 4:** Double-click the annotation to enter inline text editing mode.
- **Step 5:** As you type multi-line comments, the annotation automatically expands to a comfortable 320px minimum width to prevent awkward text wrapping.
- **Step 6:** Click anywhere outside the annotation to persist the updated dimensions and text.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Canvas SVG rendering layer attaches mouse event listeners (mousedown, mousemove, mouseup) to the .annotation-resize-grip handle.

- **Step 2:** Real-time bounding box calculation recalculates width and height CSS attributes without redrawing the entire SVG tree.
- **Step 3:** Auto-grow logic monitors input events and enforces min-width: 320px dynamic expansion.
- **Step 4:** On edit commit, serialized XML attributes are synced to the underlying /opt/unetlab/labs/<lab>.unl file via REST API.

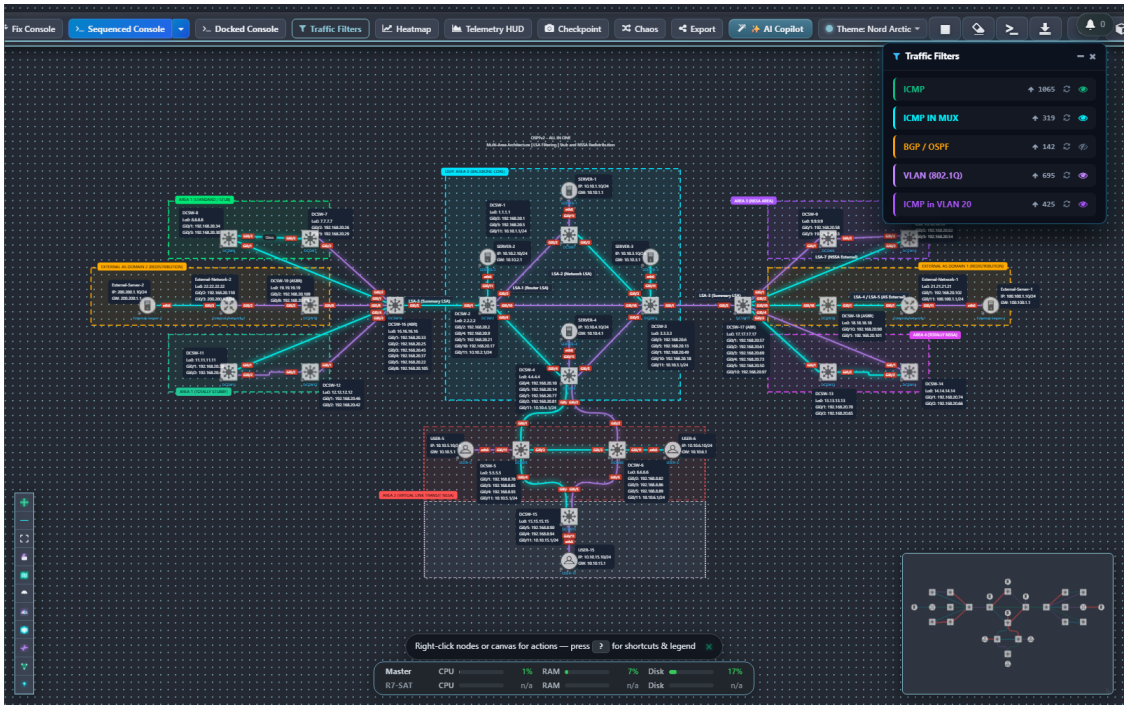


Figure: Canvas Annotation Ergonomics displaying Corner Resize Handles and 320px Auto-Grow Text Labels

32. Web Console Right-Click Context Menu & Terminal Controls [GUI: YES]

UI & System Placement: Lab Workbench Web Console > Right-Click Menu (#console-ctx-menu)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click any running node on the canvas to open the HTML5 Web Console terminal session.
- **Step 2:** Right-click anywhere inside the terminal window to open the custom dark-themed context menu.
- **Step 3:** Execute quick terminal actions: **Copy Selection**, **Paste Clipboard**, **Clear Screen**, **Zoom In (+)**, **Zoom Out (-)**, or **Preferences**.
- **Step 4:** The menu automatically intercepts browser default context menus and adapts its coordinate positioning to prevent clipping near viewport edges.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** console-init.js attaches a contextmenu event listener to the terminal DOM container.
- **Step 2:** Synthesizes clipboard actions using the asynchronous navigator.clipboard API with fallback to document.execCommand.
- **Step 3:** Coordinates screen clear events by sending ANSI escape sequence ■[2J■[H to the underlying pseudo-terminal (PTY) bridge.
- **Step 4:** Bounds-checks cursor (X, Y) offset against window dimensions to anchor submenus inside the active viewport.

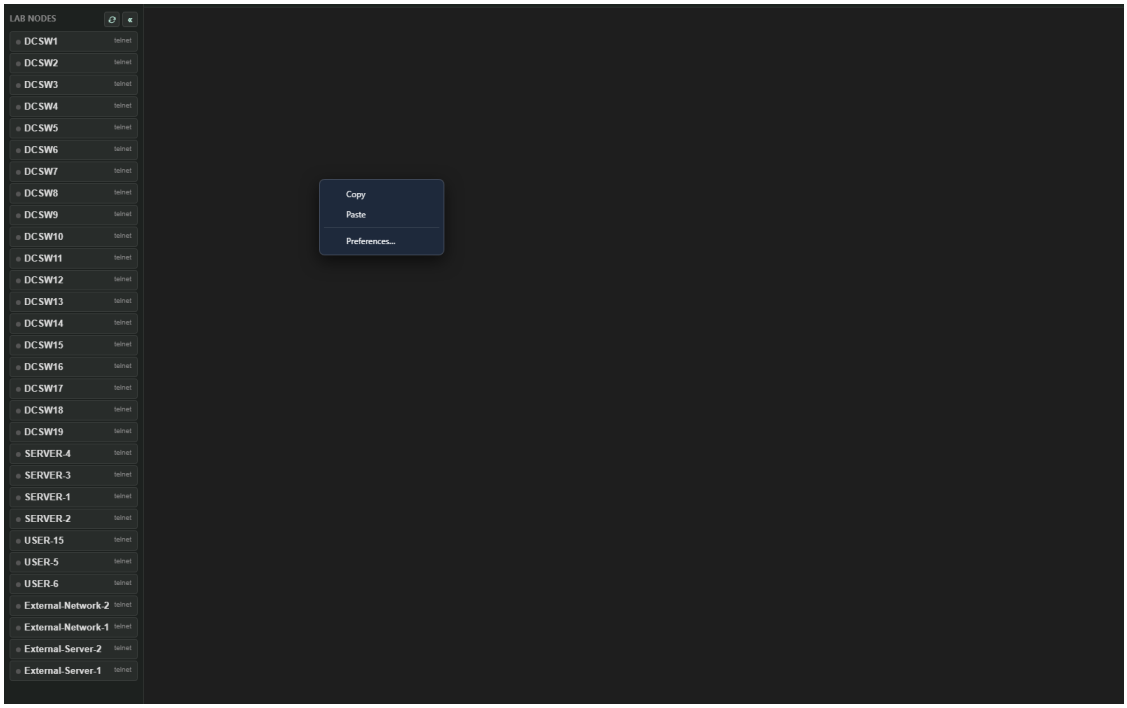


Figure: HTML5 Web Console Right-Click Context Menu for Rapid Copy, Paste, and Terminal Management

33. Terminal Preferences Modal & Custom Color Themes [GUI: YES]

UI & System Placement: Web Console Right-Click Menu > **Preferences** > Preferences Dialog (#term-prefs-modal)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open the Web Console right-click context menu and click **Preferences**.
- **Step 2:** In the Terminal Preferences modal, adjust the font size slider between **11px** and **18px**; observe the terminal text resizing in real-time.
- **Step 3:** Select from 5 professional color themes: Midnight Dark, Cyber Neon, Solarized Light, Retro Green, and Obsidian.
- **Step 4:** Click **Apply & Save**. Settings are permanently saved and will apply automatically to all future console sessions on this browser.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Preferences dialog manages state via JavaScript object { fontSize: 13, theme: 'midnight' }.
- **Step 2:** Updates xterm.js terminal instance theme options and re-fits terminal geometry via fitAddon.fit().
- **Step 3:** Emits PTY window resize event (TIOCSWINSZ) over WebSocket bridge to recalculate terminal column/row boundaries.
- **Step 4:** Serializes preferences to browser localStorage.setItem('azam_term_prefs', ...).

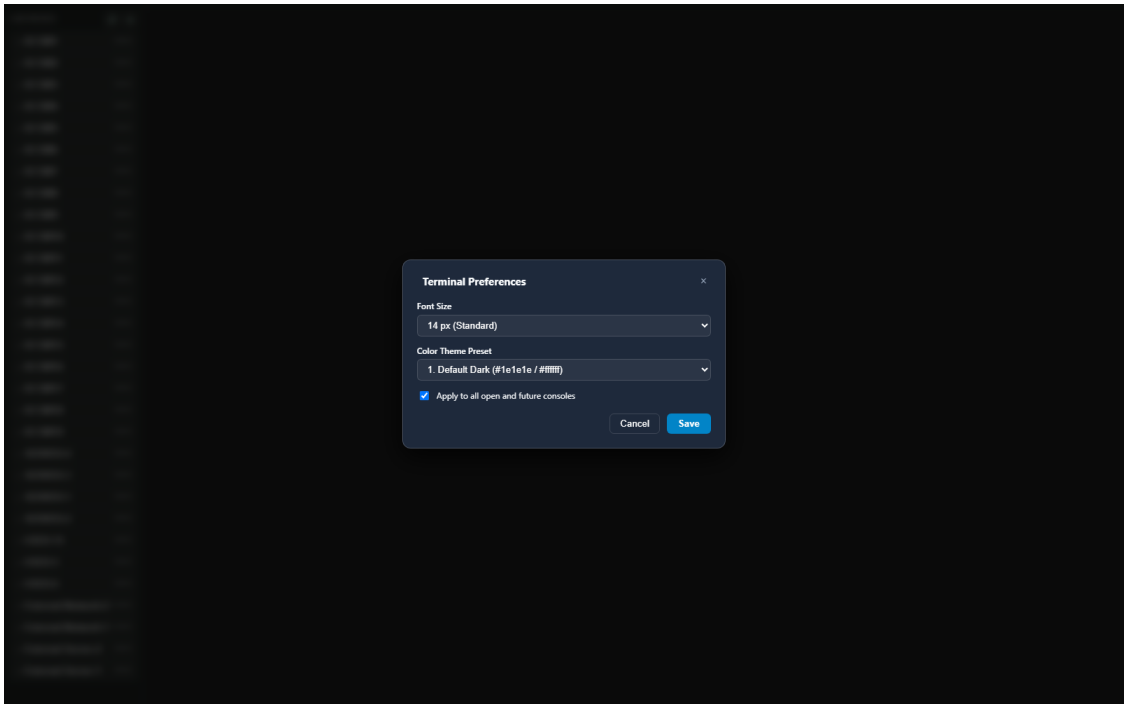


Figure: Terminal Preferences Modal featuring Dynamic Font Sizing (11px-18px) and 5 Color Themes

5. Background Daemons & Kernel Infrastructure (System Plane)

These core features operate autonomously beneath the graphical interface on both Master and Satellite nodes. They maintain hypervisor stability, self-heal system services, enable hardware-accelerated networking, and perform scheduled system maintenance. Because they run autonomously at the kernel or systemd level, they do not require an active graphical user interface.

34. VPCS 8-Client Session Multiplexer Daemon [GUI: NO - System Plane]

UI & System Placement: Linux Daemon Plane > /opt/vpcsu/bin/vpcs & /opt/unetlab/wrappers/vpcs_mux.py

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Add a virtual PC (VPCS) node to any lab topology and start it.
- **Step 2:** Connect to the VPCS console from multiple browser tabs, telnet clients, or distinct operator workstations simultaneously.
- **Step 3:** Up to 8 concurrent operators can interact with the same virtual PC console session at the same time.
- **Step 4:** Commands typed by any operator appear simultaneously on all connected consoles.
- **Step 5:** If an operator initiates a long-running command (e.g. ping 10.10.10.1 -c 1000), pressing **Ctrl+C** in any client window immediately interrupts the ping globally across all sessions.
- **Step 6:** If a 9th client attempts to connect, the multiplexer displays a polite notification (*Console is full (max 8 clients). Please try again later.*) and cleanly closes the connection.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Binary /opt/vpcsu/bin/vpcs invokes Python multiplexer wrapper /opt/unetlab/wrappers/vpcs_mux.py.
- **Step 2:** Extracts session ID from -m <session> argument and prepares a dedicated session directory /tmp/vpcs_session_{id} with 0777 permissions.
- **Step 3:** Spawns backend vpcs.real (netprobe ELF binary) with cwd set to the session directory, ensuring UID privilege drop to unl<session> succeeds config I/O checks.
- **Step 4:** Multiplexer creates a local master pseudo-terminal (PTY) and listens on TCP port 32768 + session.
- **Step 5:** Select loop multiplexes I/O: input from any client socket is written to master PTY, and master PTY output is non-blocking broadcast to all connected client sockets.
- **Step 6:** Intercepts byte 0x03 (Ctrl+C) and issues SIGINT to child process group, ensuring instantaneous cancellation.

35. 24/7 Self-Healing System Watchdog Daemon [GUI: NO - System Plane]

UI & System Placement: Linux Systemd Service > azam-watchdog.service (Autonomous 24/7 Background)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The watchdog runs autonomously as a Linux systemd service: azam-watchdog.service on both Master and Satellite nodes.
- **Step 2:** To inspect live status, open an SSH terminal and run: systemctl status azam-watchdog.
- **Step 3:** To view event logs: tail -f /var/log/azambasha-watchdog.log.
- **Step 4:** No manual interaction is needed; the service auto-starts on boot and runs continuously.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Daemon wakes up every 60 seconds and evaluates host health metrics.
- **Step 2:** Inspects essential processes; if any core service is down, triggers automated restart.
- **Step 3:** Monitors available system RAM; if free RAM drops below 5%, identifies hung QEMU processes and alerts.
- **Step 4:** Cleans up zombie processes and stale temporary lock files in /tmp to prevent hypervisor starvation.

29. MySQL Socket & Credential Auto-Healer [GUI: NO - System Plane]

UI & System Placement: Systemd Hook & Cron > /usr/local/bin/azambasha-fix-web-credentials (Master Node Background)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The healer runs automatically during boot, hourly via cron, and upon admin request on the Master Node.
- **Step 2:** To trigger manual repair from CLI, execute: azambasha-fix-web-credentials.

- **Step 3:** The script inspects database user passwords, table permissions, and socket symlinks.
- **Step 4:** Confirms with output message: [OK] Web credentials and MySQL socket successfully verified.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Checks if MariaDB unix socket exists at /var/run/mysqld/mysqld.sock; if missing, creates symlink.
- **Step 2:** Connects to database and verifies azamlabs user credentials match /opt/unetlab/html/includes/config.php.
- **Step 3:** Synchronizes Guacamole database credentials in /etc/guacamole/guacamole.properties.
- **Step 4:** Flushes MariaDB privileges (FLUSH PRIVILEGES;) and verifies read/write integrity.

30. Soft-RoCE RXE & MTU 9000 Network Accelerator [GUI: NO - System Plane]

UI & System Placement: Linux Kernel Module & Network Configuration (/etc/modprobe.d/, udev)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Activated on both Master and Satellite nodes during setup via azambasha-os-prerequisites.sh.
- **Step 2:** To inspect RDMA status, run: rdma link or ibv_devices.
- **Step 3:** To verify Jumbo Frame MTU across bridge interfaces: ip link show | grep mtu.
- **Step 4:** All virtual bridges (pnet0 through pnet9) operate with MTU 9000 for high-throughput packet emulation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Loads Linux kernel modules rdma_rxe and ib_core into kernel memory.
- **Step 2:** Binds software RoCE (RDMA over Converged Ethernet) devices to physical NIC interfaces.
- **Step 3:** Configures udev rules and /etc/network/interfaces setting bridge MTU to 9000.
- **Step 4:** Eliminates CPU packet fragmentation overhead for high-speed emulated links (e.g. 10G/40G data center fabrics).

31. Scheduled Maintenance & Storage TRIM Engine [GUI: NO - System Plane]

UI & System Placement: System Cron > /etc/cron.d/azambasha-maintenance (Nightly 03:00 UTC)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Scheduled automatically via cron on Master and Satellite nodes: /etc/cron.d/azambasha-maintenance.
- **Step 2:** Executes nightly at 03:00 UTC without disrupting running user labs.
- **Step 3:** To run on demand from CLI: azambasha-heavy-node-optimizer.sh --maintenance.
- **Step 4:** Check maintenance log at: /var/log/azambasha-maintenance.log.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Executes fstrim -av to issue TRIM commands to underlying SSD/NVMe storage, recovering deleted blocks.
- **Step 2:** Clears Linux dentry and inode pagecache buffers via sysctl vm.drop_caches=3 if system is idle.
- **Step 3:** Truncates log files in /opt/unetlab/data/Logs/ older than 14 days and compresses historical archives.
- **Step 4:** Empties orphaned sockets in /opt/unetlab/tmp/ left behind by abnormally terminated nodes.

6. Enterprise CLI Command-Line Reference & Distribution

All twenty-six AzamLabs enterprise utilities are symlinked globally in /usr/local/bin/ across both Master and Satellite nodes. This ensures that infrastructure engineers possess identical troubleshooting capabilities from SSH terminals regardless of node type.

Command / Utility	Target Feature / Scope	Typical Invocation / Options	Node Support
azam-airgap-pack	Offline Air-Gapped Bundler	azam-airgap-pack [output_dir]	Master & Satellite
azam-watchdog	24/7 Autonomous Watchdog	systemctl status azam-watchdog	Master & Satellite
azambasha-ops-api	REST API Daemon Backend	systemctl restart azam-ops-api	Master Only
azambasha-bootstorm	Anti-Bootstorm & KSM Engine	azambasha-bootstorm --lab <lab.unl>	Master & Satellite
azambasha-doctor	Diagnostic Health Suite	azambasha-doctor --full --auto-heal	Master & Satellite
azambasha-fix-web-credentials	MySQL & Guac Socket Fixer	azambasha-fix-web-credentials --fix-guac	Master Only
azambasha-heavy-node-optimizer	KSM & Hypervisor RAM Optimizer	azambasha-heavy-node-optimizer.sh --check	Master & Satellite
azambasha-cloud-transit	Cloud Transit Overlay Bridge	azambasha-cloud-transit --tailscale --connect	Master & Satellite
azambasha-install-azam-features	Symlink & Package Installer	azambasha-install-azam-features.sh [--satellite]	Master & Satellite
azambasha-fix-network-boot	Bridge & TAP Network Boot Fix	azambasha-fix-network-boot.sh	Master & Satellite
azambasha-fix-export-and-apt	Ubuntu Package Repository Fix	azambasha-fix-export-and-apt.sh	Master & Satellite
azam-link-impair	WAN QoS NetEm Impairment CLI	azam-link-impair --iface vnet0 --delay 50	Master & Satellite
azam-roce	Soft-RoCE & MTU 9000 Driver	azam-roce --status --mtu 9000	Master & Satellite
azam-cgroups	cgroups v2 Performance Manager	azam-cgroups --pid 1234 --cpu-max 50	Master & Satellite
azam-cpu-governor	Linux CFS Bandwidth Governor	azam-cpu-governor --throttle 50	Master & Satellite

7. Future VM Automated Provisioning & Verification Guide

To guarantee 100% feature alignment on every future virtual machine, the installer scripts in the AzamLabs repository have been standardized into two automated workflows:

Future Master VM Provisioning:
1. Boot a fresh Ubuntu 20.04/22.04/24.04/26.04 server.
2. Clone the repository to /opt/azambasha.
3. Execute bash /opt/azambasha/install.sh.
4. The installer automatically provisions all web assets, MariaDB credentials, Apache/Nginx routing, compiles the REST API daemon, links all 26 CLI tools, and verifies health via Test 6.

Future Satellite VM Provisioning:
1. Boot a fresh Ubuntu server with KVM virtualization enabled.
2. Clone the repository to /opt/azambasha.
3. Execute bash /opt/azambasha/install-satellite.sh.
4. In Step 9, the script automatically triggers azambasha-install-azam-features.sh --satellite, deploying all 26 CLI symlinks, watchdog daemon, KSM tuning, and TRIM cron in headless mode.

Documentation Summary & Compliance Note:
This manual encompasses all enterprise features, NetDevOps exporters, and AIOps platform enhancements integrated into the AzamLabs platform. All API endpoints, UI placements, and background scripts adhere to the platform's non-destructive design architecture. Stale websockets, hypervisor memory thresholds, and disk structures are guarded by automated checks to guarantee continuous lab uptime across both Master and Satellite compute nodes.